

AI휴먼(AI Human) 개발자 과정

강의일차

Day 2

교과목

Python 프로그래밍

강의주제

Python 실행 흐름과 변수·자료형·객체 이해

오늘의 핵심 질문: "Python에서 값은 어떻게 만들어지고, 이름을 붙여 다시 사용할까?"

```
def fibonacci(n):
    """Generate Fibonacci sequence up to n terms."""
    if n <= 0:
        return []
    elif n == 1:
        return [0]
    elif n == 2:
        return [0, 1]

    sequence = [0, 1]
    while len(sequence) < n:
        next_val = sequence[-1] + sequence[-2]
        sequence.append(next_val)
    return sequence

# Example usage
result = fibonacci(10)
print(f"Fibonacci: {result}")

# Class-based data structure
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

head = Node(42)
```

오늘의 학습 흐름

01

09:10~09:50

전날 OT·개발환경 리뷰, 오늘 목표 확인

03

11:00~12:50

대표 실습코드 따라하기와 주석 기반 설명

05

15:00~16:50

Basic → Application → Challenge 수준별 과제

02

10:00~10:50

Python 실행 흐름, 변수, 자료형, 객체, `type()` 핵심 개념

04

14:00~14:50

단답형 테스트 및 코드 결과 예측

06

17:00~17:50

오류 리뷰, 코드 설명, 다음 일차 예고

 오늘의 산출물: 개인 프로필 출력 프로그램

오늘의 학습목표

- 1 Python 코드가 실행되는 기본 흐름을 설명할 수 있다.
- 2 변수(Variable)의 의미와 대입(Assignment)을 설명할 수 있다.
- 3 정수, 실수, 문자열, 불리언, None의 차이를 구분할 수 있다.
- 4 객체(Object)와 변수 이름의 관계를 초보자 수준에서 설명할 수 있다.
- 5 `type()`으로 값의 자료형을 확인할 수 있다.
- 6 여러 자료형을 사용해 개인 프로필을 출력할 수 있다.

Python이란?

범용 프로그래밍 언어

Python은 읽기 쉬운 문법을 강조한 범용 프로그래밍 언어이다.

다양한 활용 분야

데이터 분석, AI/ML, 웹, 자동화, 크롤링 등 다양한 분야에서 사용한다.

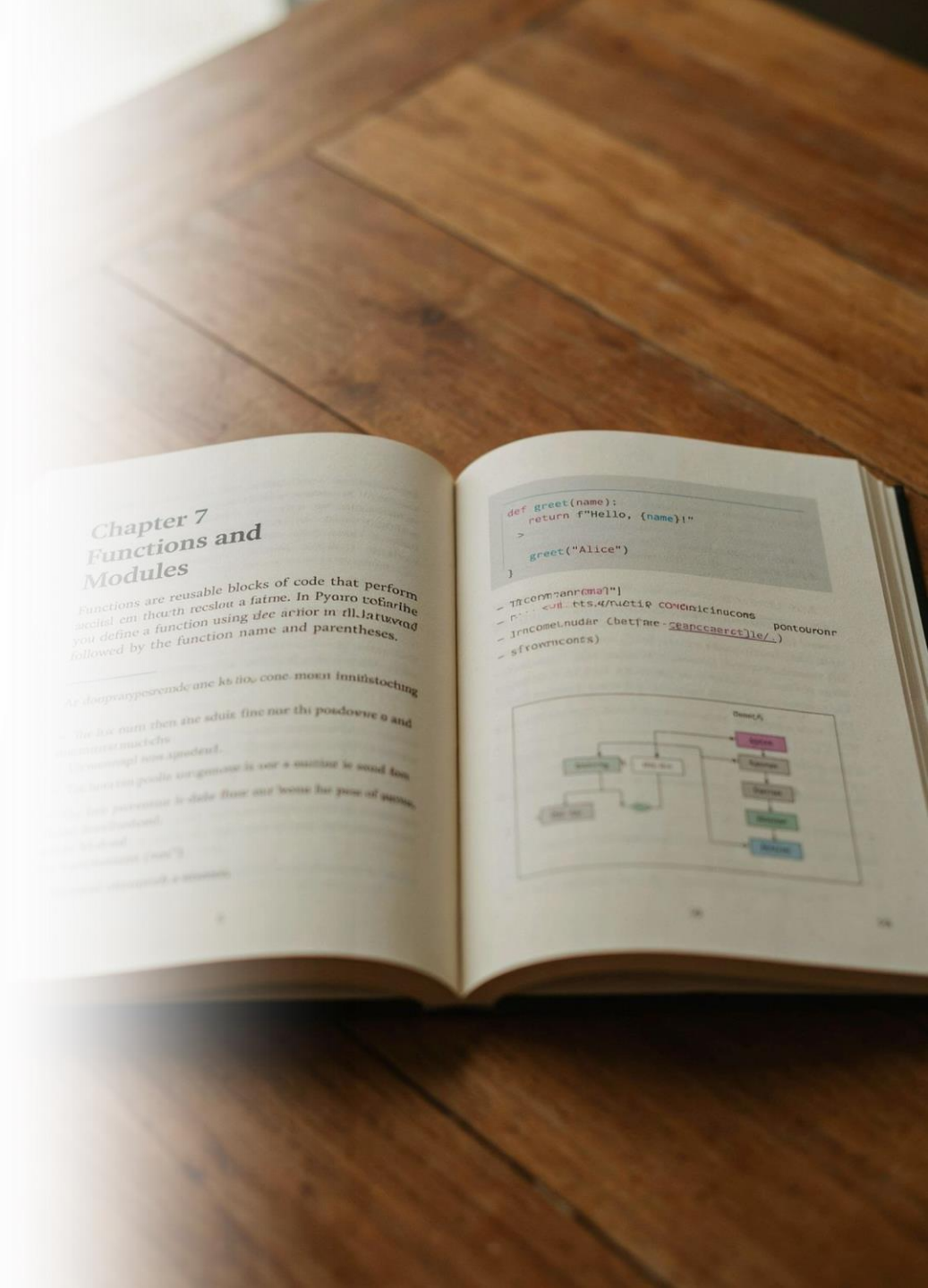
과정의 공통 기반

이번 과정에서는 이후 Pandas, 머신러닝, 딥러닝, LLM 실습의 공통 기반으로 사용한다.

핵심 이해

핵심은 "문법 암기"보다 "입력 → 처리 → 출력" 흐름을 이해하는 것이다.

용어: Python / Programming Language(프로그래밍 언어)



인터프리터(Interpreter)

Interpreter

작성한 코드를 읽고 실행하도록 도와주는 프로그램

초보자 관점

"내가 작성한 Python 코드를 컴퓨터가 수행할 수 있게 연결해 주는 실행기"

예

```
python day2_ex.py
```

CPython

Python 구현체 중 가장 널리 쓰이는 것은 CPython이다.

오늘의 집중 포인트

오늘은 내부 컴파일 과정보다 "소스코드가 실행된다"는 큰 흐름에 집중한다.

의사코드

01

Python 파일을 작성한다.

03

위에서 아래 방향으로 명령을 수행한다.

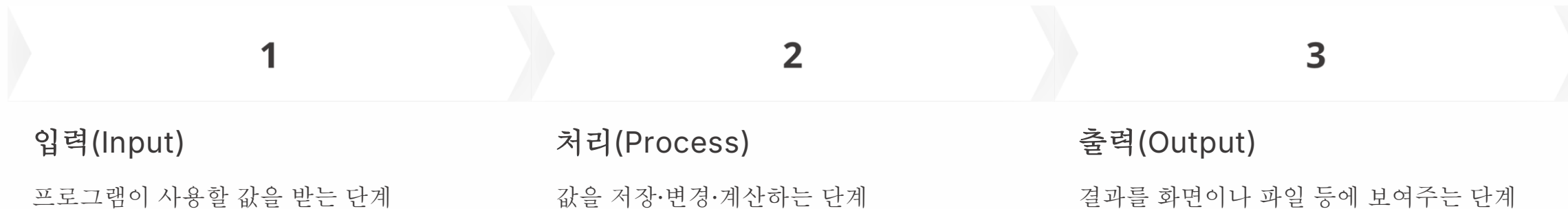
02

Python 인터프리터로 파일을 실행한다.

04

print()를 만나면 결과를 화면에 출력한다.

프로그램의 가장 기본적인 구조



Day 2는 입력보다 "값 저장과 출력"에 집중한다.

예시

```
name = "AI휴먼"  
print(name)
```

흐름: 값 "AI휴먼" 생성 → name이라는 이름으로 사용 → print()로 출력

값(Value)과 리터럴(Literal)

개념 정의

값(Value): 프로그램이 다루는 실제 데이터

리터럴(Literal): 코드에 직접 작성한 값의 표현

리터럴 종류

22

정수 리터럴

85.5

실수 리터럴

"AI휴먼"

문자열 리터럴

True

불리언 리터럴

None

값이 없음을 표현하는 특별한 값

변수(Variable)란?

Variable

데이터를 나중에 다시 사용하기 위해
이름을 붙이는 개념

초보자 이해

초보자에게는 "값에 붙이는 이름표"로
이해하면 쉽다.

정확한 관점

Python에서는 변수 이름이 특정 객체를
가리킨다고 보는 것이 더 정확하다.

예시

```
name = "홍길동"  
age = 22
```

의사코드

01

이름 값을 준비한다.

02

name이라는 이름으로 연결한다.

03

나이 값을 준비한다.

04

age라는 이름으로 연결한다.

05

필요할 때 name과 age를 다시 사용한다.

대입(Assignment)과 = 기호

핵심 개념

= 는 수학의 "같다"가 아니라 프로그래밍의 "대입한다"는 의미로 사용한다.

오른쪽의 값을 먼저 준비한 뒤 왼쪽 이름에 연결한다.

예

```
age = 22
```

읽는 순서: "22라는 값을 age라는 이름으로 사용하겠다."

주의

비교를 위한 == 는 다음 일차에 자세히 학습한다.

오늘은 = 를 대입 연산자로 이해한다.

변수 이름 규칙

- ☐ 영문자, 한글, 숫자, 밑줄(_)을 사용할 수 있다.
- ☐ 숫자로 시작할 수 없다.
- ☐ 공백을 포함할 수 없다.
- ☐ Python 예약어(keyword)를 변수명으로 사용하면 안 된다.
- ☐ 대소문자를 구분한다: age와 Age는 서로 다른 이름이다.

권장

```
student_name = "홍길동"  
user_age = 22
```

피하기

```
a = "홍길동" # 의미가 모호함
```

좋은 변수명: snake_case

Python에서는 여러 단어를 연결할 때 snake_case를 주로 사용한다.

단어 사이를 밑줄(_)로 구분한다.

예

```
student_name  
job_position  
average_score  
course_name
```

좋은 변수명 기준

변수의 의미가 드러나는가?

너무 짧거나 모호하지 않은가?

팀원이 읽어도 역할을 추측할 수 있는가?

객체(Object)란?

Python의 값

Python에서 우리가 다루는 대부분의 값은 객체(Object)이다.

Day 2 이해 수준

Day 2에서는 "값은 객체이고, 변수 이름은 그 객체를 가리킨다" 정도로 이해한다.

객체의 구성

객체는 값(Value), 자료형(Type), 정체성(Identity)을 가진다.

클래스(Class)

클래스(Class)는 객체의 설계도라는 관점으로 과정 후반 Python 파트에서 더 자세히 다룬다.

예: `age = 22`

- 22는 int 타입의 객체
- age는 그 객체를 사용하기 위한 이름

상자 비유의 장점과 한계

입문에서는 "변수라는 상자에 값을 넣는다"는 비유가 이해하기 쉽다.

하지만 Python을 더 정확히 이해하려면 "이름표가 객체를 가리킨다"가 적절하다.

초보 단계

변수 = 값을 담는 상자

한 단계 더 정확한 관점

변수 이름 → 객체

오늘의 결론

- 상자 비유로 시작해도 괜찮다.
- Python에서는 이름과 객체가 연결된다고 기억한다.

자료형(Data Type)이란?

Data Type: 데이터가 어떤 종류인지 나타내는 분류

자료형에 따라 가능한 처리 방식이 달라진다.

오늘의 핵심 자료형

int

정수

float

실수

str

문자열

bool

참/거짓

NoneType

None의 자료형

📌 **중요:** Python에서는 변수 이름보다 "변수가 가리키는 객체"가 자료형을 가진다고 이해하면 정확하다.

int - 정수 자료형

int = integer

소수점이 없는 정수 데이터를 표현한다.

예

```
age = 22  
student_count = 30  
year = 2026
```

활용 사례

나이

수강생 수

주문 수량

게시물 개수

float - 실수 자료형

float = floating-point number

소수점이 있는 수를 표현한다.

예

```
score = 85.5  
height = 172.3  
temperature = 27.8
```

활용 사례

평균 점수

온도

비율

측정값

⚠ 주의: 컴퓨터의 실수 표현은 내부적으로 근사값일 수 있으며 이는 추후 데이터 분석에서 다시 다룬다.

str - 문자열 자료형

str = string

글자, 단어, 문장과 같은 텍스트를 표현한다.

작은따옴표(') 또는 큰따옴표("")를 사용할 수 있다.

예

```
name = "홍길동"  
course = 'AI휴먼'  
job = "AI 개발자"
```

주의

"22"는 숫자처럼 보여도 따옴표 안에 있으므로 문자열이다.

bool - 불리언 자료형

bool = Boolean

참(True) 또는 거짓(False) 두 가지 상태를 표현한다.

첫 글자는 반드시 대문자이다.

예

```
is_student = True  
is_employed = False
```

활용 사례

로그인 여부

제출 여부

합격 여부

활성화 여부

None과 NoneType

None은 "현재 의미 있는 값이 없음"을 표현할 때 사용한다.

None의 자료형은 NoneType이다.

예

```
portfolio_url = None
```

의미

아직 포트폴리오 URL이 정해지지 않았다.

구분

0이나 빈 문자열과는 의미가 다르다.

확인

```
print(type(portfolio_url))
```

type() 함수

type()은 값 또는 객체의 자료형을 확인하는 내장 함수이다.

초보자에게 매우 중요한 디버깅 도구이다.

예

```
age = 22
score = 85.5
name = "홍길동"

print(type(age)) # <class 'int'>
print(type(score)) # <class 'float'>
print(type(name)) # <class 'str'>
```

동적 타이핑(Dynamic Typing)

동적 타이핑 언어

Python은 동적 타이핑 언어이다.

선언 불필요

변수 이름에 자료형을 미리 선언하지 않아도 된다.

유연한 연결

실행 중 같은 변수 이름이 다른 자료형의 객체를 가리킬 수 있다.

예

```
data = 100
print(type(data)) # int

data = "백"
print(type(data)) # str
```

⚠ 주의: 편리하지만 자료형을 잘못 예상하면 오류가 생길 수 있으므로 `type()` 확인 습관이 중요하다.

재대입(Reassignment)

이미 사용한 변수 이름에 새로운 값을 다시 연결할 수 있다.

예

```
status = "준비중"  
print(status)
```

```
status = "학습중"  
print(status)
```

출력

```
준비중  
학습중
```

📌 핵심: 코드는 위에서 아래로 실행되므로 마지막에 대입된 값이 이후 코드에서 사용된다.

여러 변수 만들기

한 프로그램에는 목적이 다른 여러 변수가 필요하다.

각 변수는 의미가 드러나는 이름으로 분리한다.

좋은 설계

한 변수에는 한 가지 의미를 담는다.

예

```
name = "김AI"  
age = 24  
job = "데이터 분석가"  
score = 91.5  
is_job_seeker = True
```

print()로 변수 출력하기

print()는 콘솔에 값을 출력하는 대표적인 내장 함수이다.

여러 값을 쉼표로 구분해 출력할 수 있다.

❏ 실습 포인트: 값 자체와 `type()` 결과를 함께 출력해 본다.

예

```
name = "AI휴먼"  
age = 22  
score = 85.5  
  
print(name)  
print(name, age)  
print(name, age, type(score))
```


f-string으로 읽기 쉬운 출력 만들기

f-string은 문자열 안에 변수 값을 삽입하는 방법이다.

문자열 앞에 f를 붙이고 변수 이름을 {} 안에 작성한다.

장점

여러 변수의 값을 사람이 읽기 좋은 문장으로 출력할 수 있다.

예

```
name = "홍길동"  
age = 22  
job = "AI 개발자"  
  
print(f"이름: {name}")  
print(f"나이: {age}세")  
print(f"희망직무: {job}")
```

대표 실습 의사코드 - 개인 프로필

문제: 이름, 나이, 희망직무, Python 경험기간, 취업준비 여부를 변수에 저장하고 출력한다.

01

프로그램 제목을 출력한다.

02

이름을 문자열로 저장한다.

03

나이를 정수로 저장한다.

04

희망직무를 문자열로 저장한다.

05

Python 경험기간을 실수로 저장한다.

06

취업준비 여부를 불리언으로 저장한다.

07

각 값을 f-string으로 출력한다.

08

type()으로 각 자료형을 확인한다.

흔한 오류와 해결 관점

1. NameError

선언하지 않은 변수 이름을 사용했을 때

예: `print(user_name)`인데 `user_name`을 만든 적이 없음

2. 따옴표 누락

문자열에 따옴표가 없으면 변수 이름으로 해석될 수 있음

예: `job = AI개발자` (X)

예: `job = "AI개발자"` (O)

3. 잘못된 변수명

예: `2name = "홍길동"` (X)

4. 자료형 혼동

22와 "22"는 서로 다른 자료형

❏ 해결 순서: 오류 메시지 확인 → 해당 줄 확인 → 변수명·따옴표·자료형 확인

오늘의 정리와 다음 일차 연결

오늘 반드시 설명할 수 있어야 하는 5가지

Interpreter

코드를 읽고 실행하도록 도와주는 프로그램

Variable

값을 다시 사용하기 위한 이름

Data Type

데이터의 종류

Object

Python에서 다루는 실제 값의 단위

type()

객체의 자료형 확인

오늘의 최종 산출물

- 개인 프로필 출력 프로그램

다음 일차 예고

- 연산자(Operator)
- input()과 사용자 입력
- 형변환(Type Casting)
- f-string을 활용한 계산 결과 출력